

Introducció a la programació i a Python

Algorísmica

Grau d'Enginyeria Informàtica

Mireia Ribera, ribera@ub.edu

Programació en Python

En aquest tema introduïm conceptes bàsics de programació que exemplifiquem amb el llenguatge Python.

Els contingut d'aquesta assignatura es complementen amb els donats a l'assignatura de Programació I, que treballa conceptes similars i els exemplifica en llenguatge Java.

L'assignatura d'algorísmica centra el focus en la ideació dels algorismes però com que els problemes es realitzen en Python **és molt recomanable al principi de curs familiaritzar-se amb el llenguatge** i l'assignatura *ofereix molts materials de suport*.

Vegeu el notebook [Referencies.ipynb](#)

Programació en Python I: Conceptes clau

- 1 Instrucció, variable, literal i expressió. Referències a variables. Comentaris
- 2 Definir, mostrar o retornar resultats.
- 3 Funció i mètode. Paràmetres. Cridar o invocar
- 4 Assignacions. Iteracions. Condicionals
- 5 Ús de ChatGPT o IA generativa

Python: variables (I)

- ❑ Les variables són **espais de memòria** on es guarden dades.
- ❑ Cada variable té un **identificador**.

Per exemple puc guardar un nom de persona i després recuperar-lo, podem guardar un valor i després fer operacions ...

Exemple:

- La variable *nom* té el valor “Adrià”
- Mostra el text “Et dius” i a continuació el valor de la variable *nom*.
- La variable *numero* té el valor 5.
- Mostra el resultat de multiplicar la variable *numero* * 10.

Python: variables (II)

- ❑ Les diferents dades no es guarden de la mateixa manera a l'ordinador. No és el mateix guardar un número enter, que un decimal, que una lletra, que una paraula, és el que s'anomena **tipus de les dades**.
- ❑ Les variable també tenen **tipus** (enter, decimals, booleà...).
- ❑ Quan creem una variable a Python no ens caldria indicar el seu tipus, però a Algorísmica **sempre indicarem els tipus de les variables** per claredat.

Python: Instrucció, variables, literals i expressions

```
# Això és un comentari: el programa no ho executa
# Instrucció: assignem un valor a una variable
edat: int = 25 # 'edat' és la variable, int vol dir integer i és el tipus, 25 és un
literal
# Una altra instrucció: imprimir per pantalla
print("Hola món!") # "Hola món!" és un literal de text (string)
# Expressió: suma d'una variable i un literal
nova_edat: int = edat + 5 # 'edat + 5' és una expressió que dona com a resultat 30
# Els operadors són +, -, *, /, // (divisió entera), ** (potència), % (mòdul)
# Mostrar el resultat
print("D'aquí 5 anys tindràs:", nova_edat)
```

Per tal que els nostres programes siguin llegibles i clars, seguirem unes pautes d'escriptura estandarditzades a Python, les directrius [PEP8](#). Vegeu el notebook [EstilPogramacio.ipynb](#).

Python

A tenir en compte! Quan escrivim un programa a Python hem de cuidar especialment dos elements:

- **Noms de variables, funcions o Identificadors:**
 - ❑ Han de començar per lletra o _ No poden contenir espais
 - ❑ Distingeixen entre majúscules i minúscules
 - ❑ Hi ha noms reservats (and, for, def)
- **Expressions:** És el codi que calcula o produeix nous valors de les dades.
 - ❑ L'expressió més simple és el literal. Pot ser un valor o una variable. Ex.a, 5, nom, "h"
 - ❑ Una expressió pot ser una combinació d'expressions més simples amb operadors.
 - ❑ Els operadors segueixen la precedència matemàtica estàndard.

Python: funcions, entrada i sortida (I)

Quan programem amb l'ordinador escrivim “*procediments computacional que prenen un (o una sèrie) de dades/valors com a entrada i generen alguna dada/valor (o sèrie de dades/valors) com a sortida.*”

Per exemple per calcular l'àrea d'un quadrat necessitem la llargada del costat, calcularem el seu quadrat i el retornarem com a sortida.

Exemple:

- Procediment `calcular_area_quadrat(mida_costat)`
- Retornar `costat * costat`



Quina és l'**entrada**? Quina és la **sortida**? De quin **tipus** són?

Python: funcions, entrada i sortida (II)

Les **dades o els valors d'entrada** que donem a un algorisme és el que anomenem **paràmetres**. Per cada paràmetre haurem d'indicar un nom de variable i el tipus.

Les **dades o valors de sortida** que ens genera l'algorisme els podem:

- ❑ **Mostrar** per pantalla (print) – els veurem
- ❑ **Retornar** (return) - vol dir que jo puc assignar a una variable el resultat de l'algorisme. Ex:

```
preu:int = calcul_IVA(100)
```

 *Pots escriure el codi corresponent a la funció de càlcul de l'àrea del quadrat i indicar els paràmetres i el seu tipus?*

Python: funcions, entrada i sortida (III)

Les **funcions** són petits algorismes que tenen entrades i sortides. A Python els posarem nom i un cop definides, les podrem “cridar” sempre que vulguem.

```
# Definició de la funció “area_quadrat”  
def area_quadrat(costat:int) -> int:  
    return costat**2  
  
area_quadrat(5)
```

 *Veus alguna diferència entre print i return?*

Python: crida de funcions

Quan Python rep la crida d'una funció, fa quatre coses:

- ❑ El programa que fa la crida se suspèn/congela en el punt de la crida.
- ❑ Els paràmetres de la funció passen a prendre els valors de la crida.
- ❑ S'executa el cos de la funció.
- ❑ Retorna el control al punt de programa posterior a la crida.

```
def sum(a:int, b:int) -> int:  
    return a + b  
def dif(a:int, b:int) -> int:  
    return a - b
```

```
def sumdif(a:int, b:int) -> int:  
    s:int = sum(a, b)  
    d:int = dif(a, b)  
    return s, d
```

```
a:int = 3  
b:int = 3  
print(sumdif(a, b))
```

Atenció! Si no cridem la funció, no s'executa!

Python: definir (def), cridar, mostrar resultats(print) , retornar resultats(return)

```
def suma(a: int, b: int) -> int #només definim, no executem
    print("la suma és ", a+b)
    return a + b

c: int = suma(5,7) # cridem la funció, c pren el valor del return

def dos_valors(a: int) -> tuple[int, int]:
    return a, a+2

d: int
e: int
d, e = dos_valors(4) # cridem la funció, c, d prenen el valor del return
print(c,d,e)
```

Vegeu el notebook Funcions.ipynb.

Python: funció i mètode

1 FUNCIO: bloc de codi que retorna un resultat

```
def suma(a: int, b: int) -> int:
    return a + b

resultat: int = suma(3, 4) #Crida
```

2 MÈTODE: forma part d'un objecte o tipus. La sintaxi és diferent.

```
text: str = "hola món"
majuscles: str = text.upper()
print("Text en majúscules ",
      majuscles)
```

3 FUNCIO SENSE RETORN: encara que retorni None és una funció.

```
def saluda() -> None:
    print("Hola! Encantat de conèixer-te!")
    # Només mostra el resultat per pantalla

saluda() # Crida la funció però no retorna res
```

4 FUNCIO DEL LENGUATGE: són funcions de Python, no les definim nosaltres

```
len(text)

# len retorna la llargada en caràcters d'una
cadena de caràcters.
```

Python: paràmetre, crida o invocació

```
# [1] Definim una funció amb dos paràmetres
def suma(a: int, b: int) -> int:
    return a + b

# [2] Definim una funció sense paràmetres
(funció que no necessita informació de fora)
def missatge() -> str:
    return "Hola!"
```

```
resultat1:int = suma(3, 4)
# passem 3 i 4 directament
print("Cas 1:", resultat1) #

x: int = 10
y: int = 5
resultat2: int = suma(x, y)
# passem el contingut de x i y
print("Cas 2:", resultat2) #

text: str = missatge()
# no necessita informació extra
print("Cas 3:", text)
```

Ara et toca... (1)



Escriu una funció que retorni la multiplicació de tres nombres enters
Escriu una altra funció que mostri per pantalla 5 vegades "hola" i retorni "molt de gust"
Usa el mètode .count de str per saber quantes "a" hi ha en una paraula

Python: variables

Recordem: les variables són **espais de memòria** per guardar-hi valors. Cada variable és d'un **tipus** i accepta uns **determinats valors**.

Aprenem noves coses: per donar valors a les variables podem fer

- ❑ **assignacions directes** al codi, amb literals, expressions o funcions,
- ❑ **assignar simultàniament valors a més d'una variable,**
- ❑ **demanar-li els valors a l'usuari** (assignacions d'entrada).
 - Sempre “llegirem” una cadena i l'haurem de convertir al tipus que volem.
 - La conversió es pot fer amb eval o directa

Python: assignacions

```
# --- ASSIGNACIONS AMB LITERALS ---
```

```
numero: int = 10      # enter
preu: float = 19.99    # nombre decimal
nom: str = "Maria"     # text (string)
es_major: bool = True  # booleà
```

```
# --- ASSIGNACIONS AMB EXPRESSIONS ---
```

```
suma: int = 5 + 3      # resultat = 8
producte: int = numero * 2 # resultat = 20
preu_final: float = preu * 1.21 # aplica IVA
nom_complet: str = nom + " López" # concatenació
gran: bool = numero > 5 # comparació
```

```
# --- ASSIGNACIÓ AMB CRIDA A FUNCIÓ ---
```

```
longitud_nom: int = len(nom)
# len() retorna la longitud de la cadena
valor_abs: int = abs(-42)
# abs() retorna el valor absolut
maxim: int = max(10, 3, 25)
# màxim de tres valors
```

```
# --- ASSIGNACIÓ MÚLTIPLE ---
```

```
x: int          # tipus
y: int
x, y = 1, 2      # assigna 1 a x i 2 a y

a: str          # tipus
b: str
c: str
a, b, c = "A", "B", "C"
```

Python: assignacions d'entrada. Necessitat de conversió

```
# --- TEXT (string) ---
nom1: str = input("Introdueix el teu nom: ")
nom2: str = eval(input("Introdueix el teu nom entre cometes: "))

# --- ENTER (int) amb conversió directa ---
edat1: int = int(input("Introdueix la teva edat: "))
edat2: int = eval(input("Introdueix la teva edat: "))

# --- DECIMAL (float) amb conversió directa ---
altura1: float = float(input("Introdueix la teva altura en metres: "))
altura2: float = eval(input("Introdueix la teva altura en metres: "))

print(nom1,nom2,edat1,edat2,altura1,altura2)
```

Python: iteracions

Per **no repetir una instrucció** moltes vegades podem fer servir **l'estructura de control de la iteració** (bucles, loops, ...).

Hi ha dos tipus d'iteracions bàsics a Python:

- ❑ Iteracions **for**: recorren una llista de valors establerta
- ❑ Iteracions **while**: es repeteix el codi mentre es compleix una condició.

Python: iteracions for: range(start (0), stop (end), step (1))

```
# Recorregut sobre una llista
Colors: list[str] = ["vermell",
                    "blau", "verd"]

for color in colors:
    print(color)

# Recorregut sobre una paraula
for lletra in "Hola":
    print(lletra)
```

```
# Amb range(): números del 0 al 4
for i in range(5):
    print(i)

# Amb range(): del 2 al 10 saltant
de 2 en 2
for i in range(2, 11, 2):
    print(i)
```

Python: iteracions while

```
# Demanem un número fins que sigui positiu
num: int = int(input("Introdueix un número positiu: "))
while num <= 0:
    num = int(input("Introdueix un número positiu: "))
print("Has introduït:", num)
```

For: sabem quan s'aturarà. Millor per a recorreguts.

While: desconeixem quan s'aturarà, depèn dels valors en el moment d'execució. Millor per a cerques.

Python: conditionals

Per **executar codis alternatius** segons el valor d'una condició podem fer servir **l'estructura de control condicional**.

Aquestes estructures tenen diversos elements:

- ❑ Sempre comencen per **if**: a if indiquem la condició inicial i posem el codi que s'executarà si es compleix
- ❑ Quasi sempre tenen un **else**: a else posem el codi que s'executarà si la condició NO es compleix.
- ❑ Poden tenir un o més **elif (else + if)**: amb noves condicions per proveir de més alternatives.

Vegeu el notebook [Control.ipynb](#)

Python: conditionals (estructures de control)

```
edat: int = 20
```

```
if edat < 18:
```

```
    print("Menor d'edat")
```

```
elif edat < 65: # opcionals, 0 o tants com vulguem
```

```
    print("Adult")
```

```
else:
```

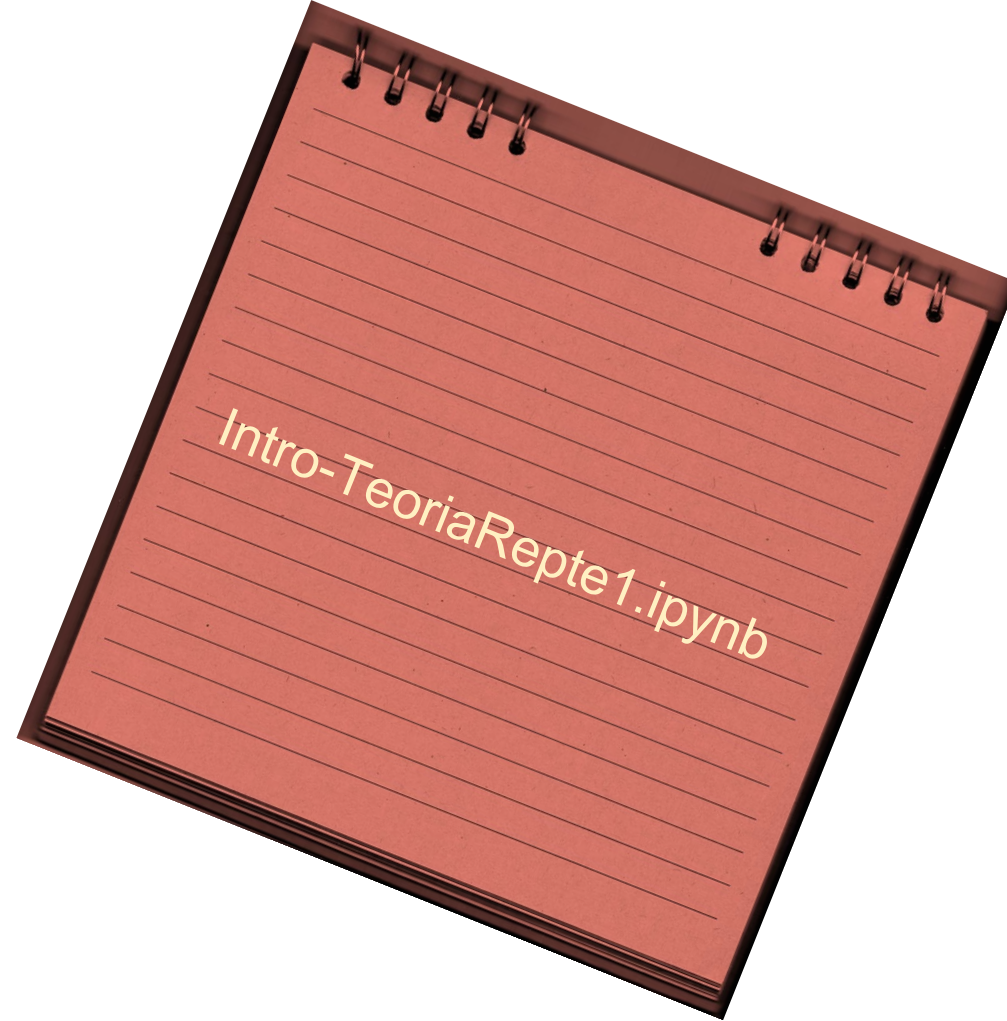
```
    print("Sènior")
```

Ara et toca... (2)



Escriu una funció que vagi demanant un número a l'usuari fins a que aquest entri 100. Per a cada número que entri ha de mostrar un missatge a pantalla: has entrat un número molt petit (≥ 0 i < 50), has entrat un número molt gran (≥ 50), has entrat un número negatiu (< 0). Quan l'usuari entri 100 ha de mostrar totes les desenes de 0 a 100, 100 inclòs, i ja no ha de demanar cap més número.

Més difícil...



Escriu una funció que retorni el factorial d'un número entrat per paràmetre. No usis cap llibreria de Python.

Ajuda de Python

- Quan no recordem alguna funció o mètode de Python podem consultar:
 - [La documentació oficial del llenguatge](#)
 - [Els cursos de W3C](#)
- ChatGPT ens pot ajudar però sempre cal verificar i entendre el què està fent.
Als exàmens ni a les proves de laboratori no es pot usar.

Ús de IA generativa a l'assignatura

La IA generativa actualment és un suport important en la programació professional. Com a estudiants de la UB podeu disposar de Copilot a Visual Studio gratuïtament. La IA pot ajudar en les fases inicials de programació, a depurar o explicar codi i a fer canvis de versions.

- Com a enginyers informàtics **haureu de tenir la capacitat de REVISAR els resultats de la IA.**
- Quan usar-la i com?
 - A casa, primer ho intento jo. Quan acabo he garantir que **la meva autonomia.**
 - Si no trobo la solució després d'intentar-ho li pregunto. Per entendre algun algorisme que no veig clar, li puc demanar ajuda. Per trobar casos extrems li puc demanar ajuda
 - Quan la IA m'ha ajudat o m'ha respost **reviso** tot pel meu compte, provo i jugo. Entenc a fons.
- **No es permet l'ús d'IA a cap prova avaluada. Tampoc als laboratoris**

Python: codi amb IA 1 (amb errors!)

```
def percentatge(part: float, total: float) -> float:  
    """Retorna el percentatge que representa 'part'  
    sobre 'total'"""  
    return part / total * 100
```

Detecta l'error de la funció anterior
Què ha passat? El programa peta?

La IA pot ser un suport, però tu has d'entendre a fons el que fa el programa.

Python: codi amb IA 2 (amb errors!)

```
def es_major_d_edat(edat: int) -> bool:  
    """Comprova si una persona és major d'edat"""  
    return edat > 18
```

Detecta l'error de la funció anterior
Què ha passat? El programa peta?

La IA pot ser un suport, però tu has d'entendre a fons el que fa el programa.

Python: codi amb IA 3 (amb errors!)

```
def text_a_majuscules(text: str) -> str:  
    """Converteix un text a majúscules"""  
    text.upper()  
    return text
```

Detecta l'error de la funció anterior

Què ha passat? El programa peta? Upper modifica "text"?

La IA pot ser un suport, però tu has d'entendre a fons el que fa el programa.

Notebooks de suport



Disposes de diversos notebooks per revisar i practicar el que hem vist fins ara:
EstilProgramacio, EntradaSortida.ipynb. Control.ipynb, Funcions.ipynb,
PythonPractica1.ipynb

Gràcies i recapitulació



MIREIA RIBERA | ACCESSIBILITAT DIGITAL
EXPERIÈNCIA D'USUARI
VISUALITZACIÓ DE DADES



UNIVERSITAT DE
BARCELONA

Recapitulació

- ✍️ En el codi usarem diferents noms i cal que indiquin clarament que fan o què són.
- 💬 Els comentaris ajuden
- ⬅️ ➡️ A Python definim l'input amb els paràmetres i l'output per pantalla o com a retorn
- 📊 Podem donar valor a una variable des d'un literal, una expressió o amb una funció.
- 🔄 Si volem repetir codi farem servir iteracions
- 🌿 Si volem executar una part de codi o una altra segons alguna variable o expressió farem servir condicionals.
- 🚫 🤖 No es permet l'ús de la IA a les proves d'avaluació.

Python II: Conceptes clau

- 1 Operadors relacionals i booleans
- 2 Cadenes de caràcters (strings)
- 3 Col·leccions de dades: llistes, diccionaris i tuples
- 4 Biblioteques externes

Python: operadors relacionals

A banda dels operadors aritmètics que hem anat veient, Python ens ofereix uns altres operadors que ens permeten comparar expressions i que donen com a resultat un valor booleà (*True* o *False*). Són els operadors relacionals:

- < # menor que
- <= # menor o igual que
- == # igual que, **no confondre amb l'assignació!**
- > # major que
- >= # major o igual que
- != # diferent a

Python: operadors booleans (I)

A més, també podem combinar expressions lògiques amb els operadors booleans. De nou, el seu resultat és un valor booleà (*True* o *False*)

Són:

- ❑ **and**, *a and b*, el seu resultat és *True* si i només si els dos operands són *True*
- ❑ **or**, *a or b*, el seu resultat és *True* si com a mínim un dels dos operands és *True*
- ❑ **not**, *not a*, el seu resultat és invers al valor del seu operand.

Python: Operadors booleans (II)

```
a:bool = True
b:bool = False
c:bool = a and b
print(c)
print(a and b or c)
print(a or (not b) and c)
```

Prova de canviar els valors d'a, b i c i observa el resultat.

Python: cadenes de caràcters

Són un tipus de dades més complex perquè es guarda com un objecte, com una seqüència de caràcters indexats.

Aquesta estructura ens permet llegir un o més caràcters determinats amb els índexs.

```
a = 'Hola'
print(a)
type(a)

s = "Hello Bob"

x = 2

print(s[0], s[x], s[8-2])

> H l B
```

Vegeu el notebook Cadenes.ipynb de Laboratori

Python: cadenes de caràcters - immutables

Les cadenes de caràcters són **immutables**, és a dir no puc fer assignacions directes a cadascun dels seus caràcters.

```
# Les cadenes són immutables. El següent codi donarà error  
paraula = "hola"  
paraula[0] = "e" # Error
```

Python: cadenes de caràcters i entrada

Quan demanem dades per teclat estem rebent strings i si volem tractar-los com a altres dades hem d'usar eval o fer una conversió explícita amb int, bool o float, per exemple.

```
a: str = 'Hola'  
print(a)  
type(a)
```

```
nom = input("Quin és el teu nom?")  
edat = int(input("Quina és la teva edat?"))
```

Python: cadenes de caràcters - codificació

L'ordinador emmagatzema els caràcters com a nombres. A cadascun li assigna un codi. Inicialment els caràcters es van codificar amb la taula ASCII (American Standard Code for Information Interchange), però actualment es codifiquen amb la taula UniCode que inclou tots els idiomes del món.

```
print(chr(65))  
print(ord("A"))
```

Python: cadenes de caràcters – mètodes i funcions

Python ens ofereix operadors, mètodes i funcions per facilitar el treball amb cadenes, els veureu amb més detall a les sessions de laboratori, aquí posem només alguns exemples:

```
print("jo" in "tu i jo")
text:str = "    Un exemple "
print(text[1:15:2])
llista: list[str] = text.split()
print(llista)
text_recuperat:str = " ".join(llista) print(text_recuperat)
print("text original:",text,". Amb strip:", text.strip(),". Però:",text)
print(text[4].isupper(),text[4].islower(),text[4].isalpha())
print(text.upper(),".Però:",text)
print(text.lower(),".Però:",text)
```

Ara et toca... (3)



Escriu una funció que donat un text i una paraula indiqui si aquesta paraula es troba en el text (usa "in"). La funció ha de ser prou robusta per trobar paraules encara que estiguin amb accent o amb majúscules o minúscules.

Python: col·leccions de dades

Sovint els problemes necessiten treballar amb una col·lecció de “coses”, sovint no sabem quantes.

Per exemple: les dades d'un experiment, els estudiants d'un curs, els ingredients d'una recepta...

En alguns exercicis ens han aparegut llistes de valors, aquesta és la col·lecció més bàsica.

Anem a veure'n algunes característiques importants

- ❑ Una llista és una **seqüència ordenada** de coses.
- ❑ Les llistes poden contenir qualsevol tipus de dades i en una mateixa llista hi poden haver diferents tipus. Són **inhomogènies**.
- ❑ Són **mutables**, és a dir, es poden canviar sobre la mateixa estructura.

Vegeu el notebook [Colleccions.ipynb](#) i [Llistes.ipynb](#)

Python: llistes – funcions i mètodes

Python ens ofereix operadors, mètodes i funcions per facilitar el treball amb llistes, els veureu amb més detall a les sessions de laboratori, aquí posem només alguns exemples:

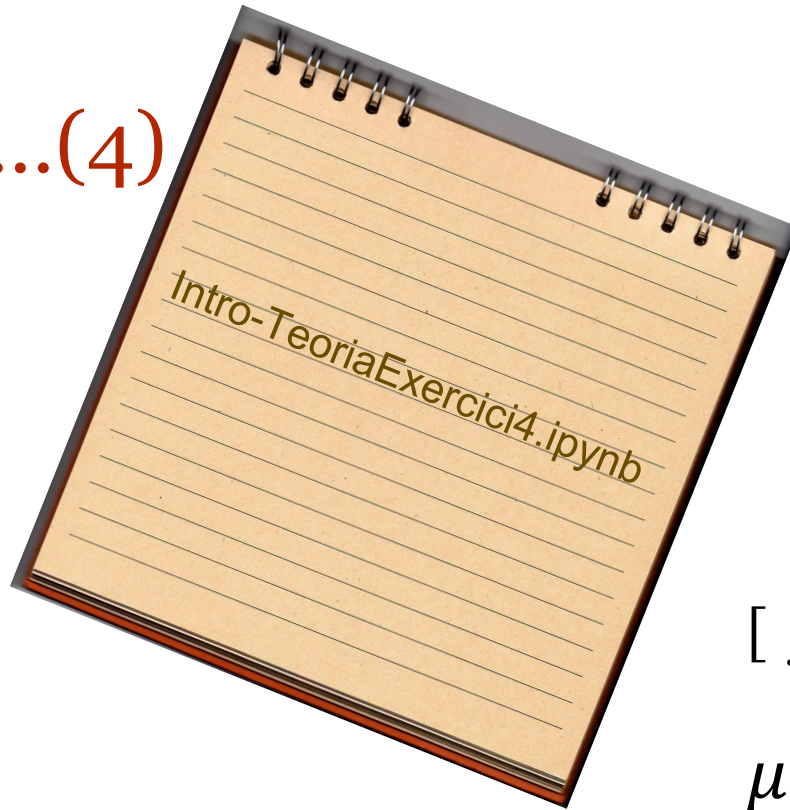
```
l1:list[Any] = []
l1.append("gat")
l1.append(2)
print(l1)
print("reverse:",l1.reverse(),l1)
l2:list[int] = [4, 2, 8, 9, 1, 2, 4, 7]
print("sort:",l2.sort(),l2)
print("count:",l2.count(4),l2.count(8))
print("index:",l2.index(9))
l2.remove(8)
print("Després del remove:",l2)
print("pop:",l2.pop(5))
print("Després del pop:",l2)
print("in:",9 in l2)
```

Python: matrius com a llistes de llistes

Python no té un tipus específic per a matrius, però les podem expressar com a llistes de llistes.

```
m = [[1,2,3],[4,5,6],[7,8,9]]  
print(m[0])  
print(m[1][1])
```

Ara et toca...(4)



$$\mu = \frac{1}{n} * \sum_{i=1}^n X_i \quad \sigma = \sqrt{\frac{\sum_{i=1}^n (X_i - \mu)^2}{n - 1}}$$

Escriu una funció que demani l'edat dels membres de la teva família o de la teva colla i que finalment mostri la mitjana (μ) i la desviació estàndard (σ).

Python: diccionaris

Els diccionaris són una altra col·lecció molt útil. Ens permeten guardar parelles de valors en les que un d'ells funciona com a índex i permet recuperar l'altre valor.

A laboratori veureu amb més detall el funcionament dels diccionaris, aquí posem només un exemple.

```
persones = {"33998877": "John Cooper", "88996644": "Elizabeth Carl", "34896745": "Erik Peters"}
```

```
def digues_nom(dni: str) -> str:  
    return persones[dni]
```

```
digues_nom("88996644")  
>> 'Elizabeth Carl'
```

Python: referències i còpia profunda

A Python cada variable té associada una referència, segons el tipus de variable aquestes referències tenen un funcionament diferent.

```
a:int = 4
b:int = 5
c:int = 6
print(id(a), id(b), id(c))
d:str = "banana"
e:str = "banana"
print(id(d),id(e))
f:list[int] = [1, 2, 3]
g:list[int] = [1, 2, 3]
print(id(f),id(g))
```

Vegeu el notebook [Referencies.ipynb](#)

Python: clonatge de variables

Si dues variables apunten a la mateixa referència, les modificacions poden ser perilloses. A les col·leccions, per evitar efectes indesitjats, no les copiarem directament, les **clonarem**.

```
a:list[int] = [1, 2, 3]
b:list[int] = a
print(a,b,id(a),id(b))
b[2] = 4
print(a)
```

```
>> [1, 2, 3] [1, 2, 3] 1673421612160 1673421612160
>> [1, 2, 4]
```

```
a:list[int] = [1, 2, 3]
b:list[int] = a[:] # b ja no apunta a "a", n'ha fet un "clon"
print(a,b,id(a),id(b))
b[2] = 4
print(a)
```

```
>> [1, 2, 3] [1, 2, 3] 1673421611072 1673420642368
>> [1, 2, 3]
```

Ara et toca... (5)



Refés l'exercici 3 anterior, usant diccionaris per relacionar cada vocal amb accent amb la vocal corresponent sense accent.

Python: tuples

El darrer tipus de col·lecció que veurem són les tuples.

Les tuples són molt semblants a les llistes però són **immutablees**. Això fa que el seu tractament informàtic sigui més eficient.

```
t: tuple = (1,2,3,4)
print(t[3])
print(t[2:4])
t1: tuple = ("gat",4,"gos")
print(t1)
t.append(5)
t[0]="canari"
```

Python: biblioteques externes

A banda dels tipus, mètodes i funcions bàsics de Python, aquest llenguatge disposa de múltiples biblioteques (en. *libraries*) externes que podem “importar” i usar. Hi ha biblioteques per treballar amb gràfics, per treballar amb dates...

En particular la biblioteca “math” ens serà força útil a l’assignatura.

```
import math

a:float = 4.5
b:int = 3
c:int = 2
d:int = 4

print(math.ceil(a))    # retorna l'enter més proper major que el nombre donat
print(math.cos(0.5))   # retorna el cosinus. Math inclou totes les funcions trigonomètriques
print(math.sqrt(d))    # calcula l'arrel quadrada
```

Gràcies i recapitulació (2a part)



UNIVERSITAT DE
BARCELONA

Recapitulació (2a part)

✅ ❌ Hi ha operadors que retornen valors booleans.

📄 Les cadenes de caràcters i les llistes tenen un funcionament similar, però les cadenes són immutables.

📦 Cal triar la col·lecció que més ens convé: una llista, un diccionari o una tupla.

🔧 Si volem una funció molt específica segurament hi ha una biblioteca externa que la inclou.